Real-Time Object Detection and Tracking with YOLOv4-tiny: A Frame-by-Frame Performance Whitepaper

Aman Ali | Student ID. b7/2035 | AI - Internship

Task 4: Mini Project, Computer Vision Object Detector and Frame-by-Frame Segmenter Pipeline

Abstract

This whitepaper presents a computer vision pipeline that detects, labels and tracks objects in video and measures its behaviour frame by frame. It combines the pre-trained YOLOv4-tiny detector (run through the OpenCV deep neural network module), a centroid tracker and a live per-class count overlay. On a 795-frame pedestrian video (768 x 576 pixels, 10 FPS) on a CPU-only machine, the pipeline averaged 114.0 ms per frame (8.8 FPS), about 88% of the video's native rate. Latency was stable (standard deviation 8.3 ms) and did not depend on the number of objects in view. No ground-truth annotations were available, so the evaluation covers speed and output statistics, not detection accuracy.

1. Introduction

Real-time object detection and tracking underpins traffic monitoring, surveillance and robotics. A usable system must find the objects in each frame, keep their identity over time, and run fast enough to keep up with the video. The project brief asks for a pipeline that extracts and tracks target objects in a video stream, overlays dynamic per-class counts, and evaluates frame-by-frame inference metrics. The brief allows OpenCV or a pre-trained deep learning model such as YOLO; this implementation uses a pre-trained YOLO model, which is one of the two permitted options.

2. System Overview

The pipeline processes the video one frame at a time through five stages:

1. **Acquisition:** OpenCV reads the next frame from a video file, a webcam, or an automatically downloaded sample video.

2. **Detection:** the YOLOv4-tiny network returns bounding boxes, class labels and confidence scores.
3. **Tracking:** each detection is matched to an object from the previous frame or given a new identifier.
4. **Overlay:** boxes, identifiers, labels and a per-class count line are drawn on the frame.
5. **Logging:** inference time, total objects and per-class counts are saved per frame, and the annotated frame is written to the output video.

Detector. YOLOv4-tiny is a compact single-stage detector (Bochkovskiy et al., 2020): one forward pass of the network predicts all boxes and classes, which makes it much faster than two-stage approaches. It is pre-trained on the COCO dataset (Lin et al., 2014) and recognises 80 everyday classes such as person, car, truck and dog. The model runs inside OpenCV, so no extra deep learning framework is needed. Frames are resized to 416 x 416 pixels, detections below 0.4 confidence are discarded, and overlapping boxes are merged by non-maximum suppression with a 0.4 threshold.

Tracker. A centroid tracker gives every object a persistent identifier. Each detection is matched to the nearest object of the previous frame; if it lies within 60 pixels it inherits that identifier, otherwise a new one is created. An unmatched object is kept for up to five frames, which tolerates short missed detections.

Overlay and interface. Every output frame carries a box and an identifier-label tag per object, and a header bar with the live per-class counts, the total and the instantaneous FPS, for example "person: 5 car: 1 | total: 6". The program is menu driven: the user downloads a sample video, supplies a video file or uses a webcam, and the detector then runs immediately.

3. Experimental Setup

The test video is the standard pedestrian sequence from the OpenCV samples, downloaded automatically by the program. It has 795 frames at 768 x 576 pixels and 10 FPS (about 79.5 seconds) and shows an outdoor scene with walking people and parked vehicles. All measurements were made on a CPU-only machine without GPU acceleration, using Python, NumPy and OpenCV. The model was loaded and warmed up before timing started, and the whole video was processed with no frame skipping. Inference time covers detection plus tracking for one frame and excludes drawing the overlay and encoding the output video.

Five quantities were recorded for every frame: inference time (ms), throughput (1000 divided by the inference time), total objects, per-class counts and, over the whole video, the number of unique identifiers created by the tracker.

4. Results

4.1 Inference time and throughput

The mean inference time over all 795 frames was 114.0 ms, an average throughput of 8.8 FPS. The median (111.4 ms) is close to the mean and the standard deviation is only 8.3 ms, so the per-frame cost is very consistent.

Statistic	Value
Frames processed	795
Mean inference time	114.0 ms
Median inference time	111.4 ms
Standard deviation	8.3 ms
Minimum / Maximum	100.7 ms / 175.4 ms
95th / 99th percentile	130.3 ms / 143.5 ms
Mean throughput	8.8 FPS

Table 1. Frame-by-frame inference time statistics.

About 84% of the frames took less than 120 ms, and only four frames (0.5%) exceeded 150 ms (Figure 1, left). These rare spikes are most likely operating-system scheduling delays rather than a property of the frame content.

4.2 Real-time capability

The test video runs at 10 FPS, so a pipeline needing 100 ms or less per frame keeps pace with it. The measured 8.8 FPS is about 88% of the native rate: close to real time, with a small delay that builds up. For a 30 FPS webcam the same speed equals about 29% of the incoming rate, so frames would have to be skipped. Live operation at 30 FPS would need a GPU or a smaller input resolution.

4.3 Independence from scene content

The number of objects per frame varied from 5 to 11 (mean 8.21, standard deviation 1.30), yet its correlation with inference time was almost zero (r = -0.018). This is expected for a single-stage detector, whose cost is dominated by the fixed-size forward pass at 416 x 416 resolution rather than by the number of objects found. Latency is therefore predictable. The mean inference time stayed between 111.3 ms and 115.2 ms in every block of 100 frames.

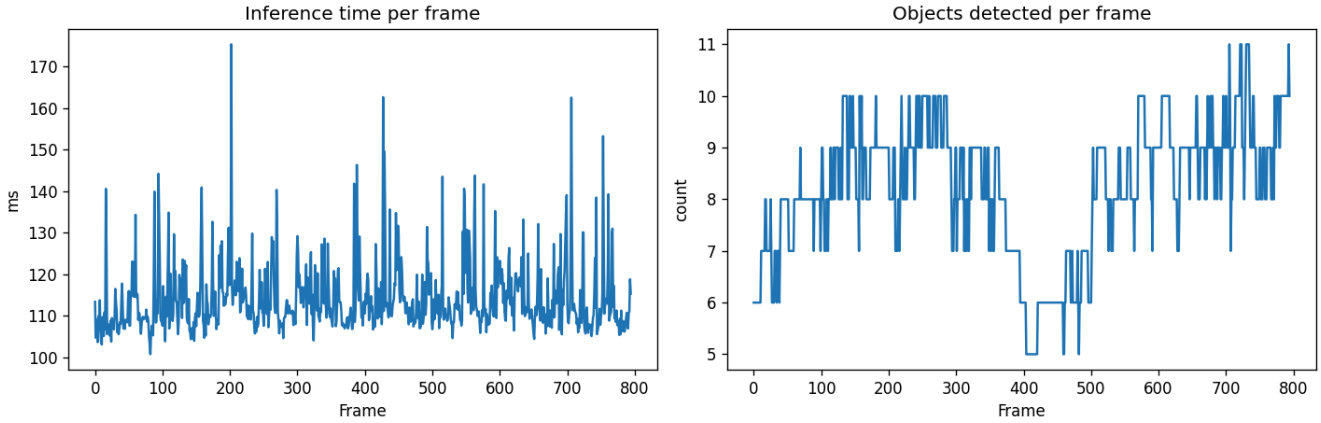


Figure 1. Inference time per frame (left) and number of objects detected per frame (right) over the 795-frame video.

4.4 Detection output

The detector produced 6,527 detections in total, 8.21 per frame, and every frame contained at least five objects. People dominate the output, with 6.21 per frame on average (maximum 9). A truck and a car are reported in almost every frame because vehicles are parked in the background of the scene for the whole video.

Class	Detections	Share of all detections	Frames containing the class
person	4,936	75.6%	795
truck	795	12.2%	795
car	792	12.1%	792
dog	3	0.05%	3
backpack	1	0.02%	1

Table 2. Detections by class over the full video.



Figure 2. Annotated output frame (frame 300) with identifiers, labels and the live count overlay. The rock at the left is wrongly labelled as a person.

4.5 Tracking behaviour

The tracker created 52 identifiers over the video, although at most nine people are visible in any frame. The simple centroid tracker therefore sometimes loses an object and issues a new identifier, for example when people cross paths or an object is missed for more than five frames. Identifiers are reliable for short-term association between consecutive frames but not for counting the distinct people in the whole video.

4.6 Comparison with a classical baseline

During development a classical baseline was prototyped: Gaussian blur, adaptive thresholding and contour extraction with minimum and maximum pixel-area bounds, followed by a polygon-corner shape classifier. It was run on the first 100 frames of the same video in the same session (Table 3).

Method	Mean time per frame	Throughput	Mean objects per frame	Unique identifiers
Classical (Gaussian blur, adaptive threshold, contours)	23.3 ms	42.8 FPS	20.6	73
YOLOv4-tiny	126.5 ms	7.9 FPS	7.4	12

Table 3. Classical baseline versus YOLOv4-tiny on the first 100 frames.

The classical method was about five times faster but reported nearly three times as many objects. Visual inspection showed that most were not real objects: regions of grass, tarmac and the building were extracted as shapes, because the method has no notion of what an object is. YOLO returned a smaller, meaningful set with a person label on each pedestrian. This is the main reason a pre-trained detector was chosen for the final pipeline. The baseline numbers come from a preliminary experiment; the delivered program uses YOLO only.

5. Discussion and Limitations

The results show a clear trade-off between speed and understanding. Classical thresholding is cheap but cannot separate meaningful objects from clutter in a real outdoor scene, whereas the deep learning detector returns labelled objects at about 114 ms per frame on a CPU, with latency that is stable and predictable.

The annotated output also shows typical weaknesses of a lightweight detector: a rock was labelled as a person in some frames, the parked white van was labelled as a truck, and a dog and a backpack appeared in a handful of frames (4 detections in total) and are likely false positives. Limitations of this study:

- **No accuracy metrics:** without annotated ground truth, precision, recall and mAP could not be computed. The counts in Table 2 describe what the detector output, not how correct it was.
- **Speed on CPU:** at 8.8 FPS the pipeline cannot keep up with 30 FPS live video on a CPU.
- **Lightweight model:** YOLOv4-tiny trades accuracy for speed and misses small or partly hidden objects more often than larger models.
- **Simple tracker:** nearest-centroid matching has no motion model or appearance features, which produced 52 identifiers for at most nine visible people.

- **Fixed vocabulary and single scene:** the model knows only the 80 COCO classes, and all results come from one video, so numbers may differ for other scenes, resolutions or hardware.

6. Conclusion and Future Work

A complete frame-by-frame detection and tracking pipeline was built with a pre-trained YOLOv4-tiny model, a centroid tracker, a live per-class count overlay and automatic metric logging. On a 795-frame pedestrian video it needed 114.0 ms per frame on average (8.8 FPS, about 88% of the video's native rate), with stable latency independent of the number of objects. Future work includes annotating frames to compute precision, recall and mAP, replacing the tracker with one that uses motion and appearance (SORT or DeepSORT), using a GPU or smaller input to reach 30 FPS, comparing newer YOLO versions, and applying Gaussian smoothing and adaptive thresholding inside each detected box to extract exact object contours.

References

1. Bochkovskiy, A., Wang, C.-Y., and Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.
2. Lin, T.-Y., et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV).
3. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
4. Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).